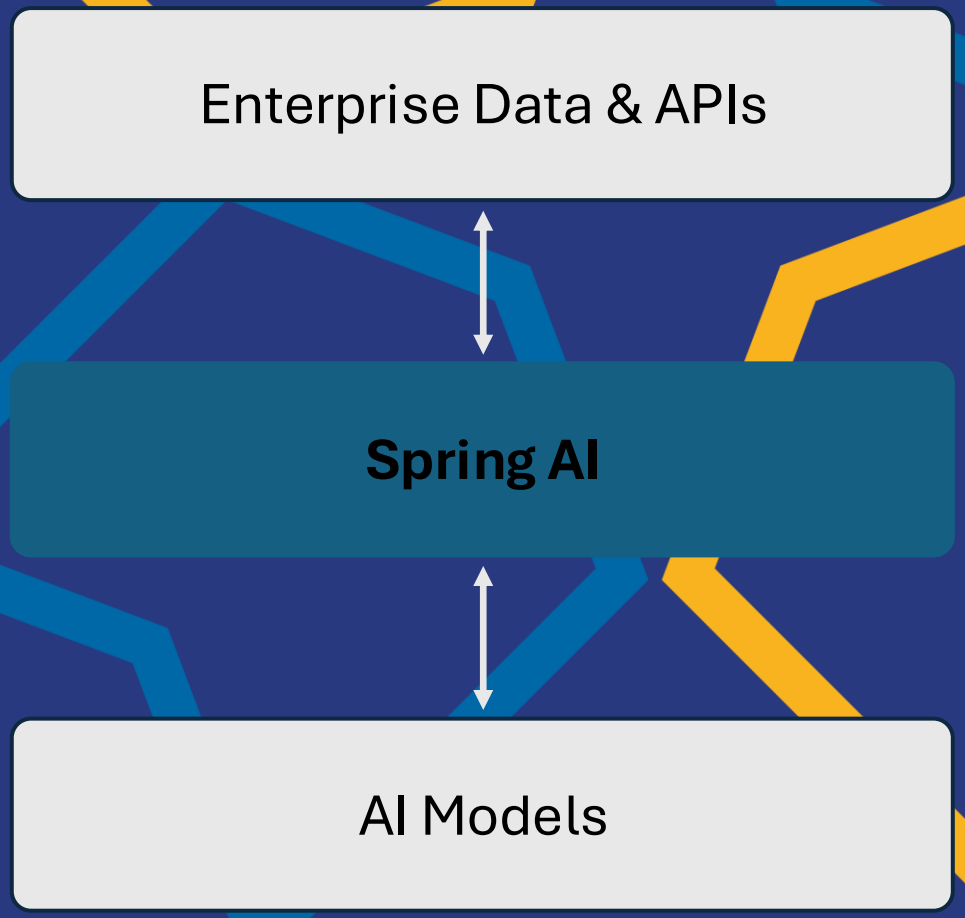
Industrial University of Ho Chi Minh City

Spring AI

Industrial University of Ho Chi Minh City

Spring AI: Connecting Enterprise Data & APIs with AI Models

Chapter 7 · Spring Boot + Spring Data + Spring Security



Agenda

1. Generative AI concept
2. Java/Spring in AI
3. Spring AI: GenAI for Spring Boot
4. Architecture & processing flow
5. Demo & Product Assistant
6. Best practices & DevOps
7. Roadmap & references

Generative AI: outputs & core technologies



Text
Emails, blogs, code, Q&A



Images
Graphic design, text-to-image

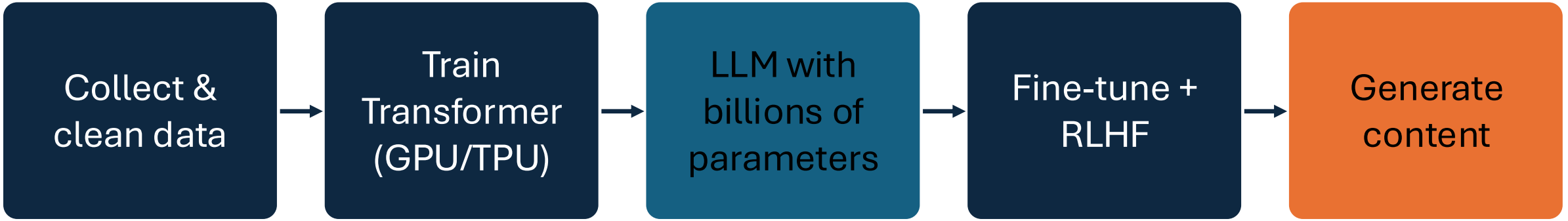


Audio / Video
Music, 3D models

Core technologies

- Transformers (GPT, BERT)
- LLMs (GPT, Claude, Gemini) **
- Diffusion models (DALL·E, Midjourney)
- Embeddings & vector space (RAG)
- Prompt engineering **

How GenAI works



- Training predicts the next token from context
- Fine-tuning and RLHF improve accuracy, usefulness and safety
- Used in chatbots, content tools, code assistants and image generators

Why Java and Spring for AI



Java for enterprise AI

- Reliable in finance, telecom, insurance
- Microservices-friendly
- Cross-platform and multithreaded

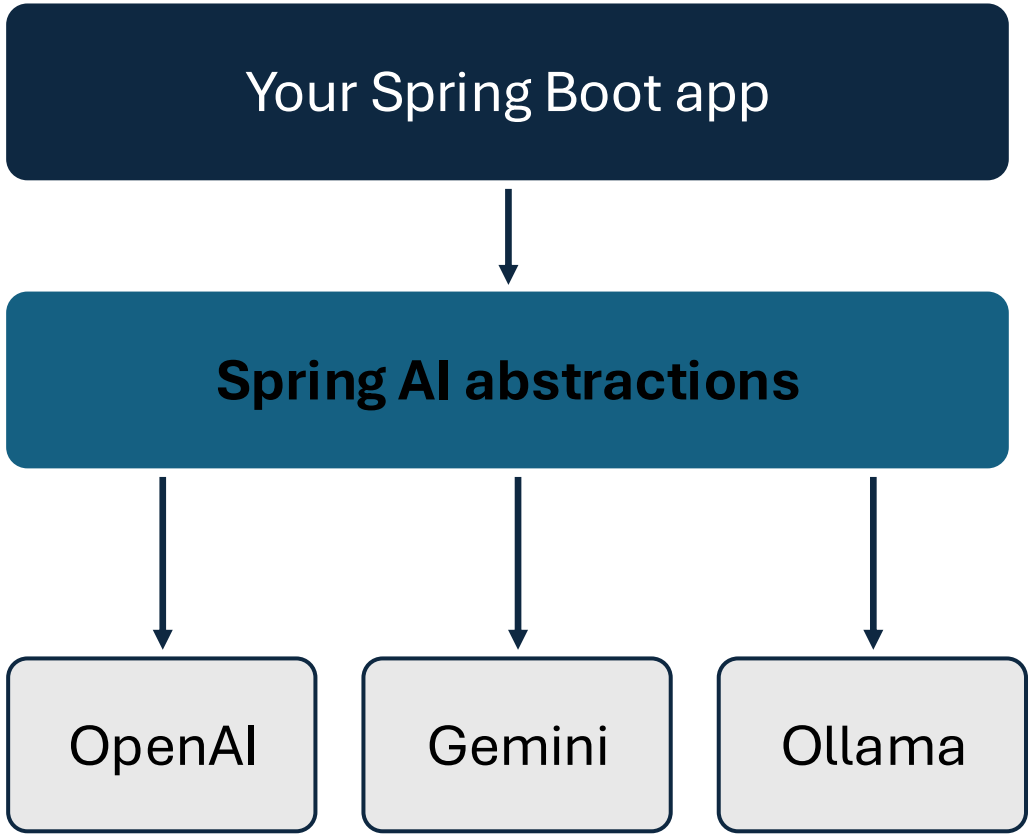


Spring Boot

- REST APIs, easy deployment
- Spring Data for your data
- Spring Security for access control

What is Spring AI

- Streamlines AI development in the familiar Spring ecosystem
- Inspired by LangChain and LlamaIndex, not a direct port
- Goal: GenAI for every language, not just Python



Swap models by configuration

Key features



ChatClient

Fluent API like RestClient



PromptTemplate

Reusable prompts with variables



Advisors & Memory

Chat history, RAG patterns



VectorStore

Semantic search on embeddings



Tool Calling

Model calls your Java methods



Structured Output

Map answers to POJOs

Supported providers

Chat & embedding models

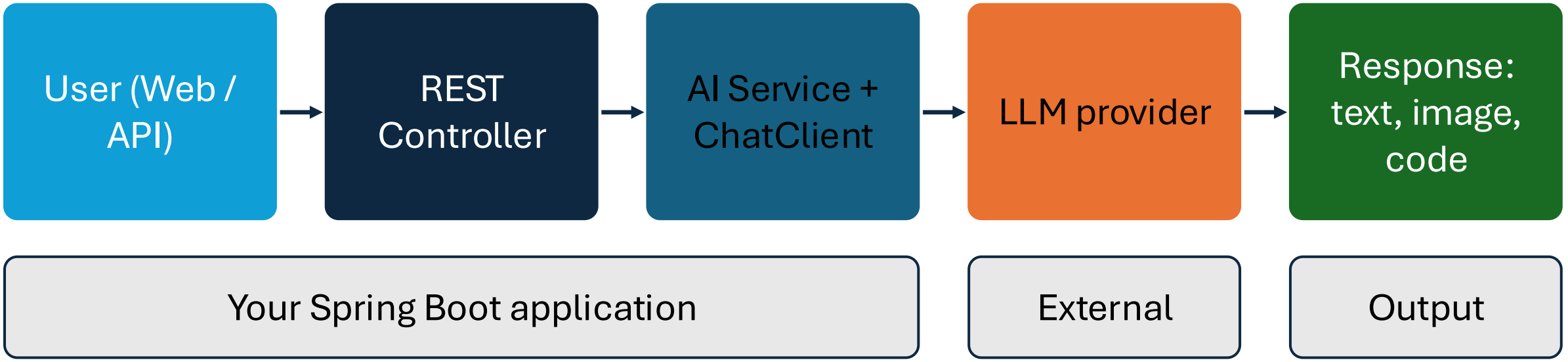
OpenAI	Anthropic	Google Gemini	Azure OpenAI
Mistral AI	Ollama (local)	Amazon Bedrock	DeepSeek

Vector databases

PGvector	Qdrant	Weaviate	Redis

Spring AI 2.0.2 is the current stable release

GenAI integration flow



ChatClient builds the prompt from a PromptTemplate, runs advisors, calls the model and returns the answer

Tool calling loop



1 Prompt + tool definitions go to the model

2 Model requests a tool call with arguments

3 Your app runs the @Tool method

4 Result returns to the model; repeat until final answer

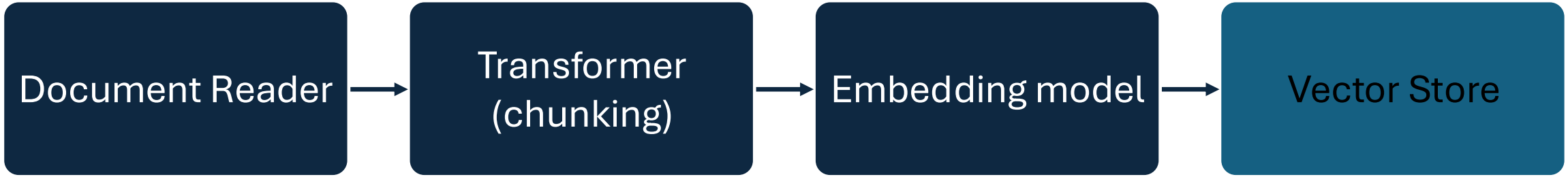
The model never touches your DB or APIs; your application executes every tool

RAG: Retrieval Augmented Generation

Query time

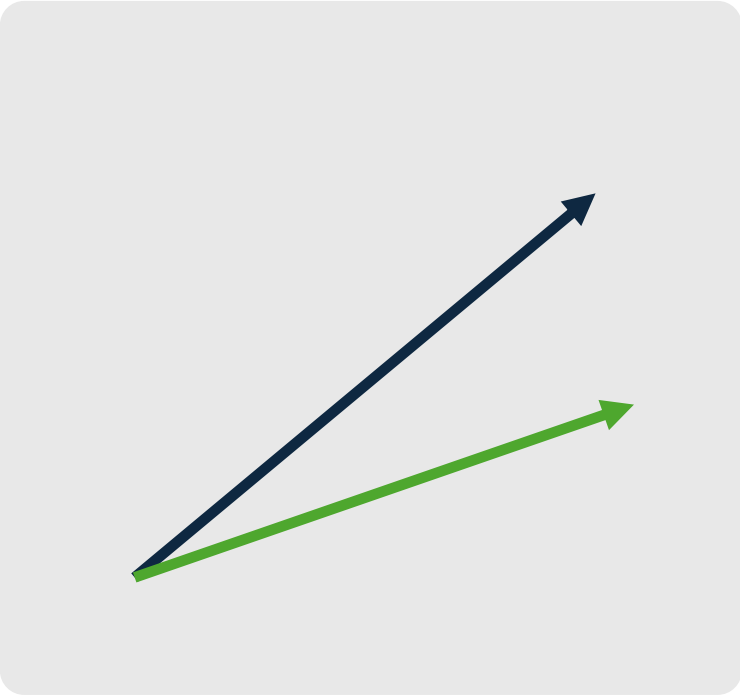


Ingestion (ETL)

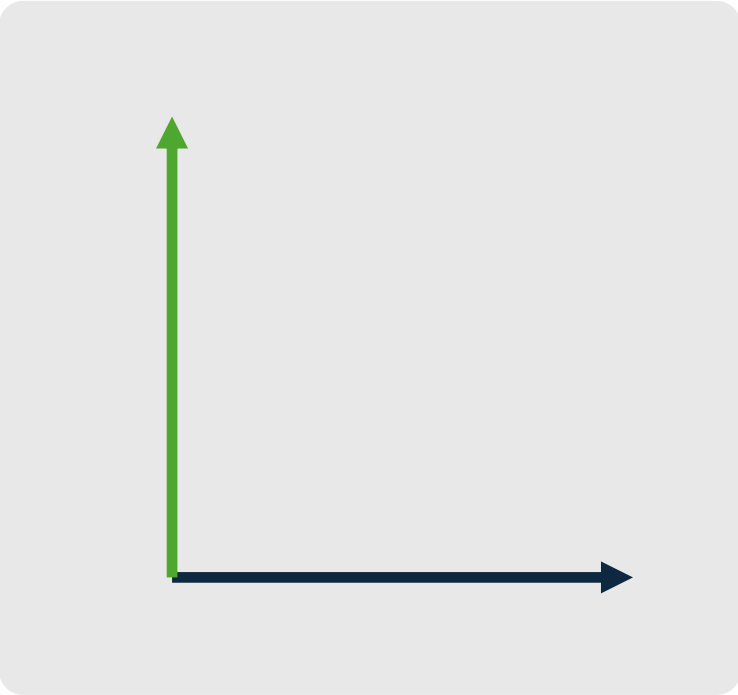


Similar text is found by cosine similarity, then added to the prompt

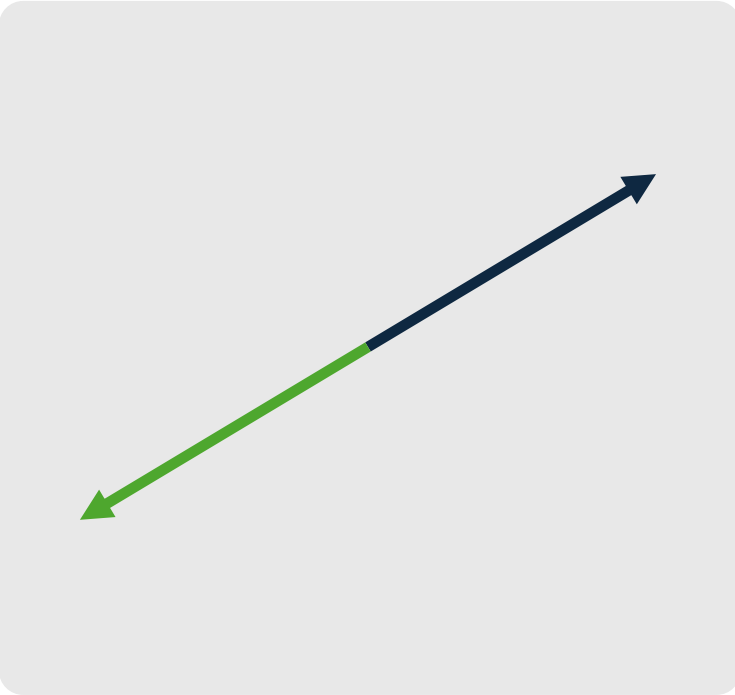
Cosine similarity



θ ≈ 0° → cos ≈ 1
Similar



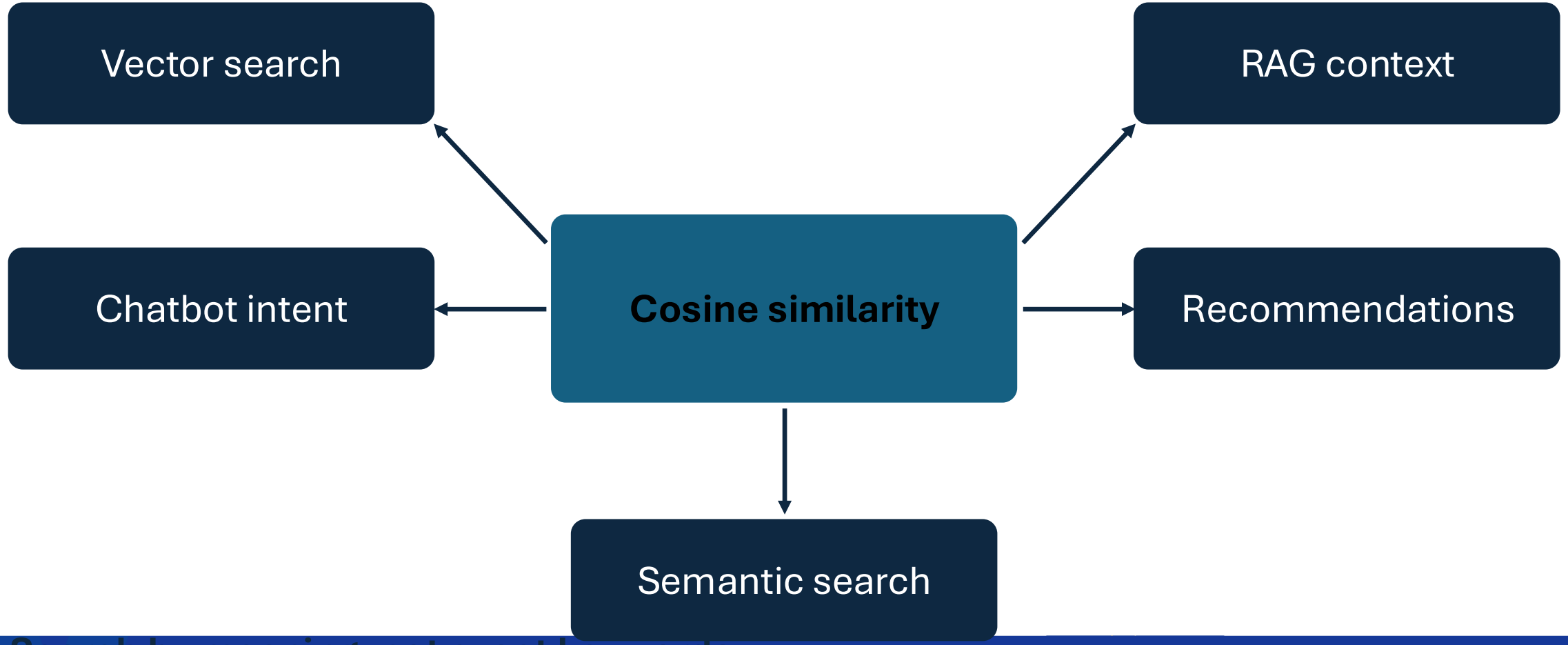
θ = 90° → cos = 0
Unrelated



θ = 180° → cos = −1
Opposite

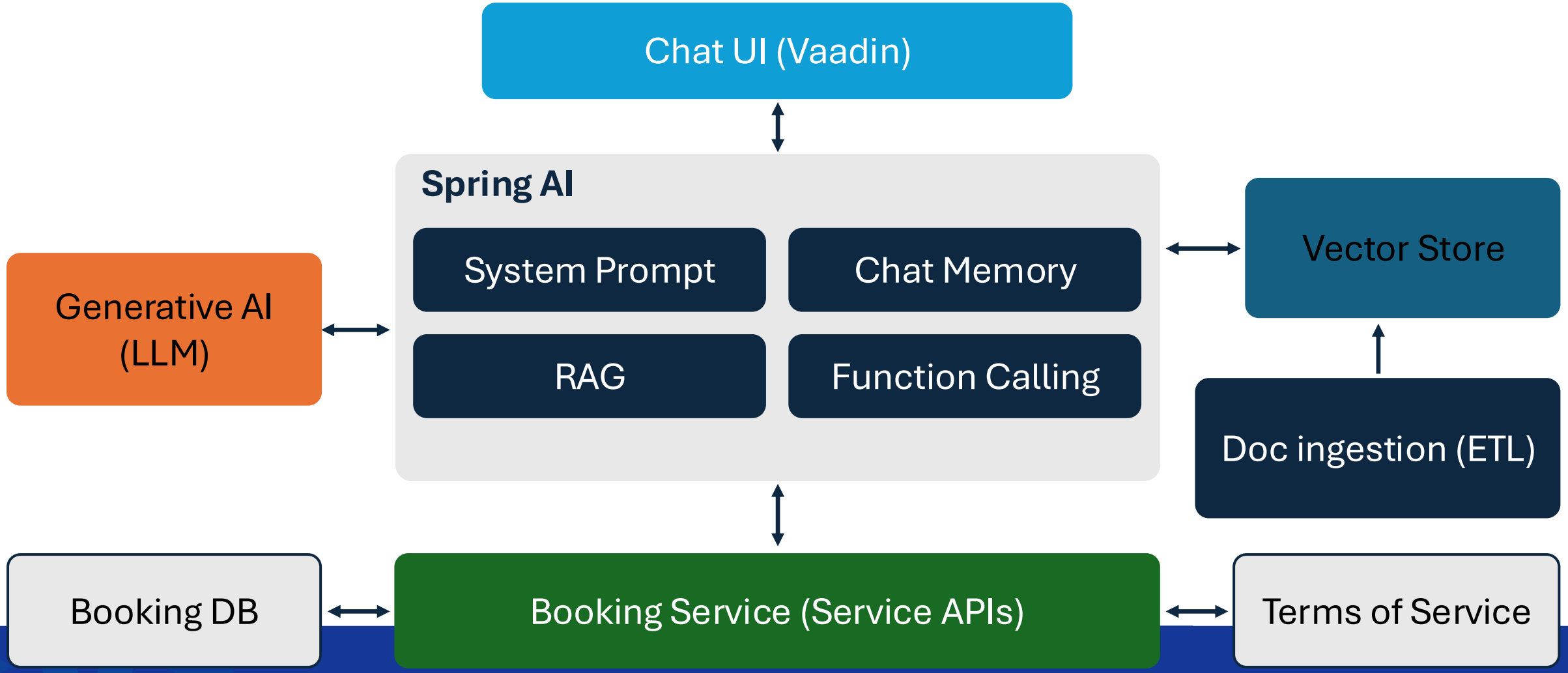
$$\cos(\theta) = (A \cdot B) / (\|A\| \|B\|)$$

Cosine similarity applications

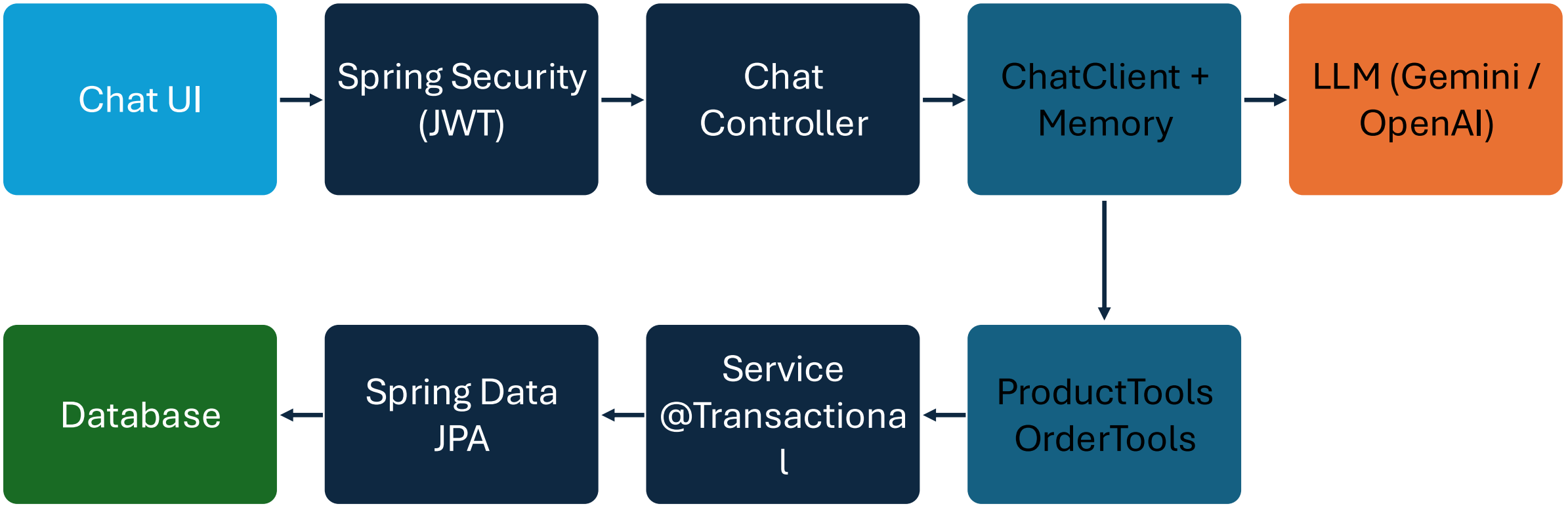


Search by meaning, not exact keywords

Demo: assistant with RAG and tool calling



Product Assistant architecture



Username comes from Authentication and travels in toolContext, never to the model

Industrial University of Ho Chi Minh City

Search and order by chat

Any laptop under 20 million VND?

3 found: Acer Aspire 5 – 15.9M VND

tool: searchProducts

Take the first one, quantity 1

Confirm 1 × Acer Aspire 5 = 15.9M VND?

memory: “the first one”

Yes, place it

Order #1024 placed successfully

tool: placeOrder

Safe ordering guardrails



Identity from the server via toolContext



Price and stock read from the DB



Validate in service: quantity, stock, role



Confirm before commit: draft order, then confirm



Limit tool calls; rate-limit the endpoint



Per-request tools for risky actions

Notes and best practices

Security & config

- API keys in env vars, Vault or CI secrets
- Never commit keys; rotate leaked ones

Prompt optimization

- Keep prompts short to save tokens
- Use templates; separate context and question

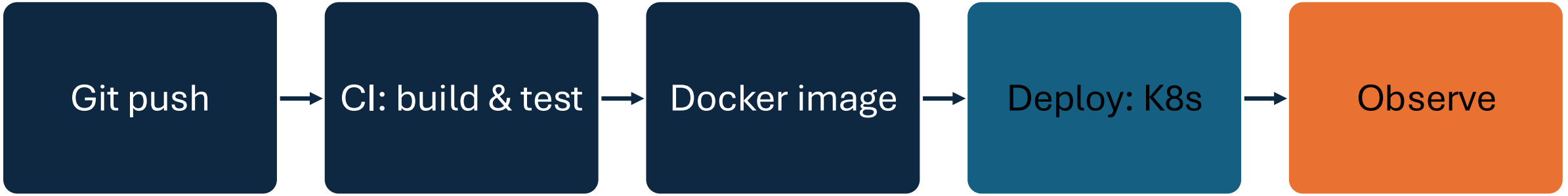
Logging & debugging

- Log input, prompt and response
- Track latency

Modular design

- Interface per AI service to swap models
- Mock AI responses in unit tests

DevOps pipeline for an AI-enabled app



Config & secrets

- API keys from Vault, K8s Secrets or CI secrets
- Dev profile uses Ollama; prod uses a cloud model

Tests & observability

- Mock the ChatModel in CI tests
- Micrometer metrics: latency, tool calls, errors

Development direction



Internal GenAI

Train or fine-tune your own model



GenAI + Workflow

AI decides inside business processes



Agents

Spring AI agents or LangChain4j



Real-time data

Kafka + RAG

References



Spring AI docs
docs.spring.io/spring-ai/reference



Qdrant
qdrant.tech



LangChain4j
github.com/langchain4j/langchain4j



Weaviate
weaviate.io



OpenAI Platform
platform.openai.com/docs



Ollama
ollama.com